LAPORAN PRAKTIKUM

JOBSHEET VI

SEARCHING

Disusun Oleh:

NAURA FADHILLA ADITYA PUTRI

254107020007

TI-1C

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

TAHUN AJARAN

2026

A. Searching/ Pencarian Menggunakan Algoritma Sequential Search

Berikut ialah hasil kode program pada class Mahasiswa20.java:

```java
package Minggu7;

class Mahasiswa20 {
    String nama;
    String nim;
    String kelas;
    double ipk;

    // constructor
    Mahasiswa20(){

    }

    Mahasiswa20(String nama, String nim, String kelas, double ipk) {
        this.nama = nama;
        this.nim = nim;
        this.kelas = kelas;
        this.ipk = ipk;

    }

    // method tampil data
    void tampilData() {
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }
```

Berikut ialah hasil kode program pada class MahasiswaBerprestasi20.java:

```java
package Minggu7;

public class MahasiswaBerprestasi20 {
    Mahasiswa20 [] listMhs = new Mahasiswa20[5];
    int idx;

    // method untuk menambah data mahasiswa
    void tambah(Mahasiswa20 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println(x: "Data sudah penuh!");
        }
    }

    // method untuk menampilkan data mahasiswa berprestasi
    void tampil() {
        for (Mahasiswa20 m : listMhs) {
            m.tampilData();
            System.out.println(x: "-------------------");
        }
    }

    // method sequential search
    int sequentialSearching(double cari) {
        int posisi = -1;
        for (int i = 0; i < listMhs.length; i++) {
            if (listMhs[i].ipk == cari) {
                posisi = i;
                break;
            }
        }
        return posisi;
    }

    // method tampil posisi mahasiswa yang dicari
    void tampilPosisi(double x, int pos) {
        if (pos != -1) {
            System.out.println("Data mahasiswa dengan IPK " + x + " ditemukan pada indeks " + pos);
        } else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
        }
    }

    // method tampil mahasiswa yang dicari
    void tampilDataSearch(double x, int pos) {
        if (pos != -1) {
            System.out.println("nim\t : " + listMhs[pos].nim);
            System.out.println("nama\t : " + listMhs[pos].nama);
            System.out.println("kelas\t : " + listMhs[pos].kelas);
            System.out.println("ipk\t : " + x);
        }
        else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
        }
    }
```

Berikut ialah hasil kode program pada class MahasiswaDemo20.java:

```java
package Minggu7;

import java.util.Scanner;

public class MahasiswaDemo20 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        MahasiswaBerprestasi20 list = new MahasiswaBerprestasi20();
        Scanner sc = new Scanner(System.in);
        int jumMhs=5;

        for (int i = 0; i < jumMhs; i++) {
            System.out.println("Masukkan data mahasiswa ke-" + (i + 1) + ":");
            System.out.print(s: "NIM: ");
            String nim = sc.nextLine();
            System.out.print(s: "Nama: ");
            String nama = sc.nextLine();
            System.out.print(s: "Kelas: ");
            String kelas = sc.nextLine();
            System.out.print(s: "IPK: ");
            String ip = sc.nextLine();
            double ipk = Double.parseDouble(ip);
            System.out.println(x: "------------------------------");
            list.tambah(new Mahasiswa20(nim, nama, kelas, ipk));
        }
        list.tampil();
        // melakukan pencarian data sequential
        System.out.println(x: "------------------------------");
        System.out.println(x: "Pencarian data");
        System.out.println(x: "------------------------------");
        System.out.print(s: "Masukkan IPK yang ingin dicari: ");
        System.out.println(x: "IPK: ");
        double cari = sc.nextDouble();

        System.out.println(x: "mnenggunakan sequential searching");
        double posisi = list.sequentialSearching(cari);
        int pss= (int) posisi;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);
    }
}
```

Berikut ialah hasil Ouput kode program pada class MahasiswaDemo20.java:

```
Masukkan data mahasiswa ke-4:
NIM: 32323
Nama: risky
Kelas: 3
IPK: 3.8
------------------------------
Masukkan data mahasiswa ke-5:
NIM: 2342
Nama: reva
Kelas: 2
IPK: 3.5
------------------------------
Nama: 12231
NIM: nawra
Kelas: 3
IPK: 3.0
------------------
Nama: 23212
NIM: rara
Kelas: 3
IPK: 2.0
------------------
Nama: 234221
NIM: dewi
Kelas: 3
IPK: 4.0
------------------
Nama: 32323
NIM: risky
Kelas: 3
IPK: 3.8
------------------
Nama: 2342
NIM: reva
Kelas: 2
IPK: 3.5
------------------
------------------------------
Pencarian data
------------------------------
Masukkan IPK yang ingin dicari:
IPK: 4
menggunakan sequential searching
Data mahasiswa dengan IPK 4.0 ditemukan pada indeks 2
nim      : dewi
nama     : 234221
kelas    : 3
ipk      : 4.0
PS C:\Users\hp\Documents\TUGAS KULYEAH\Praktikum ASD\Pratikum ASD Backup\PraktikumASD-backup->
```

Pertanyaan:

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!
   Jawab:
   - **tampilPosisi**: hanya menampilkan informasi posisi indeks hasil pencarian. Contohnya: "Data mahasiswa dengan IPK 3.5 ditemukan pada indeks 3". Jadi fokusnya pada lokasi data di array.
   - **tampilDataSearch**: menampilkan detail isi data mahasiswa yang ditemukan (NIM, Nama, Kelas, IPK). Jadi fokusnya pada isi objek yang ditemukan, bukan sekadar posisinya.

2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){
    posisi=j;
    break;
}
```

   Jawab: break digunakan untuk menghentikan perulangan (loop) secara langsung begitu kondisi terpenuhi. Dalam konteks pencarian (sequential search), begitu IPK yang dicari ditemukan, program tidak perlu lagi melanjutkan pemeriksaan elemen berikutnya.

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
   Jawab: Variabel pos berfungsi untuk menyimpan indeks array tempat data ditemukan. Dengan indeks ini program dapat menunjukkan posisi data di array, dan mengakses objek mahasiswa yang sesuai untuk ditampilkan detailnya.

4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.
   Jawab: Sequential search jika terdapat lebih dari satu data maka akan berhenti di data pertama yang cocok (karena ada break). Jadi, jika ada 2 mahasiswa dengan IPK sama, yang ditampilkan hanya mahasiswa pertama yang ditemukan.

5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?
   Jawab: Akibatnya loop akan terus berjalan sampai akhir array, Jika ada lebih dari satu data dengan IPK sama, hasil pencarian akan berakhir di data terakhir yang cocok.

B. Searching/ Pencarian Menggunakan Algoritma Binary Search

Berikut ialah hasil kode program pada class MahasiswaBerprestasi20.java:

```java
public class MahasiswaBerprestasi20 {
    // method untuk menambah data mahasiswa
    void tambah(Mahasiswa20 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println(x: "Data sudah penuh!");
        }
    }

    // method untuk menampilkan data mahasiswa berprestasi
    void tampil() {
        for (Mahasiswa20 m : listMhs) {
            m.tampilData();
            System.out.println(x: "-------------------");
        }
    }

    // method binary search
    int findBinarySearch(double cari, int left, int right){
        int mid;
        if (right >= left) {
            mid = (left + right) / 2;
            if (cari == listMhs[mid].ipk) {
                return mid;
            } else if (cari < listMhs[mid].ipk) {
                return findBinarySearch(cari, left, mid - 1);
            } else {
                return findBinarySearch(cari, mid + 1, right);
            }
        }
        return -1;
    }

    // method tampil posisi mahasiswa yang dicari
    void tampilPosisi(double x, int pos) {
        if (pos != -1) {
            System.out.println("Data mahasiswa dengan IPK " + x + " ditemukan pada indeks " + pos);
        } else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
        }
    }

    // method tampil mahasiswa yang dicari
    void tampilDataSearch(double x, int pos) {
        if (pos != -1) {
            System.out.println("nim\t : " + listMhs[pos].nim);
            System.out.println("nama\t : " + listMhs[pos].nama);
            System.out.println("kelas\t : " + listMhs[pos].kelas);
            System.out.println("ipk\t : " + x);
        }
        else {
            System.out.println("Data mahasiswa dengan IPK " + x + " tidak ditemukan");
        }
```

Berikut ialah hasil kode program pada class MahasiswaDemo20.java:

```java
package Minggu7;

import java.util.Scanner;

public class MahasiswaDemo20 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        MahasiswaBerprestasi20 list = new MahasiswaBerprestasi20();
        Scanner sc = new Scanner(System.in);
        int jumMhs=5;

        for (int i = 0; i < jumMhs; i++) {
            System.out.println("Masukkan data mahasiswa ke-" + (i + 1) + ":");
            System.out.print(s: "NIM: ");
            String nim = sc.nextLine();
            System.out.print(s: "Nama: ");
            String nama = sc.nextLine();
            System.out.print(s: "Kelas: ");
            String kelas = sc.nextLine();
            System.out.print(s: "IPK: ");
            String ip = sc.nextLine();
            double ipk = Double.parseDouble(ip);
            System.out.println(x: "------------------------------");
            list.tambah(new Mahasiswa20(nim, nama, kelas, ipk));
        }
        list.tampil();
        // melakukan pencarian data sequential
        System.out.println(x: "------------------------------");
        System.out.println(x: "Pencarian data");
        System.out.println(x: "------------------------------");
        System.out.println(x: "Masukkan IPK yang ingin dicari: ");
        System.out.print(s: "IPK: ");
        String cariIp = sc.nextLine();
        double cari = Double.parseDouble(cariIp);

        System.out.println(x: "menggunakan binary searching");
        double posisi2 = list.findBinarySearch(cari, left: 0, jumMhs - 1);
        int pss= (int) posisi2;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);
    }
}
```

Berikut ialah hasil output kode program pada class MahasiswaDemo20.java:

```
Masukkan data mahasiswa ke-3:
NIM: 23432
Nama: susi
Kelas: 2
IPK: 3.5
-----------------------------
Masukkan data mahasiswa ke-4:
NIM: 23422
Nama: anita
Kelas: 3.7
IPK: 3.7
-----------------------------
Masukkan data mahasiswa ke-5:
NIM: 2343
Nama: dika
Kelas: 2
IPK: 3.8
-----------------------------
Nama: 123122
NIM: Adi
Kelas: 2
IPK: 3.2
-------------------
Nama: 23133
NIM: ila
Kelas: 2
IPK: 3.1
-------------------
Nama: 23432
NIM: susi
Kelas: 2
IPK: 3.5
-------------------
Nama: 23422
NIM: anita
Kelas: 3.7
IPK: 3.7
-------------------
Nama: 2343
NIM: dika
Kelas: 2
IPK: 3.8
-------------------
-----------------------------
Pencarian data
-----------------------------
Masukkan IPK yang ingin dicari:
IPK: 3.5
menggunakan binary searching
Data mahasiswa dengan IPK 3.5 ditemukan pada indeks 2
nim      : susi
nama     : 23432
kelas    : 2
ipk      : 3.5
PS C:\Users\hp\Documents\TUGAS KULYEAH\Praktikum ASD\Pratikum ASD Backup\PraktikumASD-backup->
```

Pertanyaan:

1. Tunjukkan pada kode program yang mana proses divide dijalankan!
   Jawab: Proses divide terjadi saat baris ini dijalankan, divide sendiri digunakan untuk menghitung nilai Tengah atau mid.

   ```
   mid = (left + right) / 2;
   ```

   Array dibagi dua, lalu fokus pencarian diarahkan ke kiri atau kanan tergantung nilai cari.

2. Tunjukkan pada kode program yang mana proses conquer dijalankan!
   Jawab: Proses Conquer terjadi data baris ini dijalankan, Conquer sendiri digunakan untuk menentukan rekursi ke kiri/kanan

   ```
   return findBinarySearch(cari, left, mid - 1);
   lse {
   return findBinarySearch(cari, mid + 1, right);
   ```

3. Apa fungsi left, right, dan mid?
   Jawab:
   - Left : batas awal pencarian (indeks paling kiri).
   - Right: batas akhir pencarian (indeks paling kanan).
   - Mid: titik tengah yang digunakan untuk membandingkan nilai cari dengan data di tengah.

4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?
   Jawab: Binary search tidak akan bekerja dengan benar.
   Alasannya: algoritma binary search mengasumsikan data sudah terurut (ascending atau descending). Kalau tidak urut, hasil pencarian bisa salah atau tidak ketemu.

5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai
   Jawab: Dengan kode sekarang (yang mengasumsikan ascending), hasil pencarian tidak sesuai. Untuk menyesuaikan dengan descending, ubah kondisi perbandingan:

```
if (cari == listMhs[mid].ipk) {
    return mid;
} else if (cari > listMhs[mid].ipk) { // perhatikan tanda
    return findBinarySearch(cari, left, mid - 1);
} else {
    return findBinarySearch(cari, mid + 1, right);
}
```

6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
   Jawab: Jika left > right, berarti pencarian sudah melewati batas array tanpa menemukan data. Oleh karena adanya kode return -1 yang menandakan data tidak ditemukan.

```
if (right >= left) { ... }
return -1; // tanda data tidak ada
```

7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.
   Jawab:
   Di MahasiswaDemo20, ganti:

```java
public class MahasiswaDemo20 {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println(x: "Masukkan jumlah mahasiswa: ");
        int jumMhs = sc.nextInt();
        sc.nextLine();  // membersihkan newline
        MahasiswaBerprestasi20 list = new MahasiswaBerprestasi20(jumMhs);
```

   Lalu di MahasiswaBerprestasi20, ubah array agar fleksibel:

```java
public class MahasiswaBerprestasi20 {
    Mahasiswa20 [] listMhs;
    int idx;
    // constructor
    MahasiswaBerprestasi20(int jumMhs) {
        listMhs = new Mahasiswa20[jumMhs];
        idx = 0;
    }
```

   Berikut ialah hadil output modifikasi jumlah mahasiswa dibuat dinamis dengan input user:

```
Masukkan jumlah mahasiswa: 3
Masukkan data mahasiswa ke-1:
NIM: 123321
Nama: radit
Kelas: 3
IPK: 3.4
-----------------------------
```